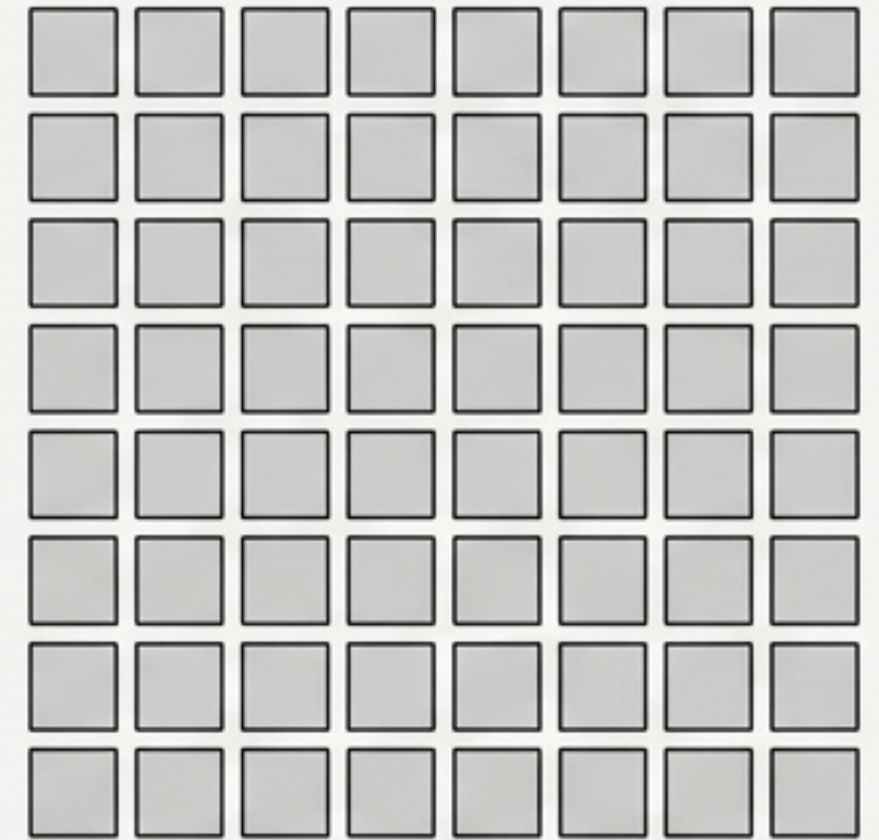


JavaScript Numbers: A Deconstruction

How every `Number` is built from the ground up, according to the IEEE 754 standard.

In JavaScript, Every Number is a 64-Bit Floating-Point Number.

- Unlike many other languages, JavaScript doesn't have distinct integer and floating-point types.
- Every number, from 1 to 3.14, is stored in the same format: a double-precision 64-bit binary format, as defined by the **IEEE Standard for Floating-Point Arithmetic (IEEE 754)**.
- This single, precise blueprint dictates how numbers are stored, calculated, and why they sometimes behave in unexpected ways.



The First Step: Separating Integer from Fraction

To convert a decimal number into binary, we must handle the integer and the fractional parts separately. Let's deconstruct our case study number: **1920.75**.
IBM Plex Sans Regular

1920.75

Integer Part

1920

IBM Plex Mono

Fractional Part

0.75

IBM Plex Mono

Converting the Integer (1920) with Repeated Division.

We repeatedly divide the integer by 2 and record the remainder. The binary result is the sequence of remainders read from bottom to top.

Division	Remainder
1920 / 2 = 960	0
960 / 2 = 480	0
480 / 2 = 240	0
240 / 2 = 120	0
120 / 2 = 60	0
60 / 2 = 30	0
30 / 2 = 15	0
15 / 2 = 7	1
7 / 2 = 3	1
3 / 2 = 1	1
1 / 2 = 0	1



111100000000

Converting the Fraction (0.75) with Repeated Multiplication.

For the fractional part, we repeatedly multiply by 2 and record the integer part of the result. The binary fraction is the sequence of integers read from top to bottom.

Multiplication	Full Result	Integer Part (Binary Digit)
0.75×2	1.5	1
0.5×2	1.0	1

Result: .11

Normalizing the Binary into Scientific Notation

Combining our parts gives us the full binary representation. To prepare it for the IEEE 754 format, we normalize it into binary scientific notation, which always starts with `1.` followed by the mantissa.

Decimal	1920.75
---------	---------

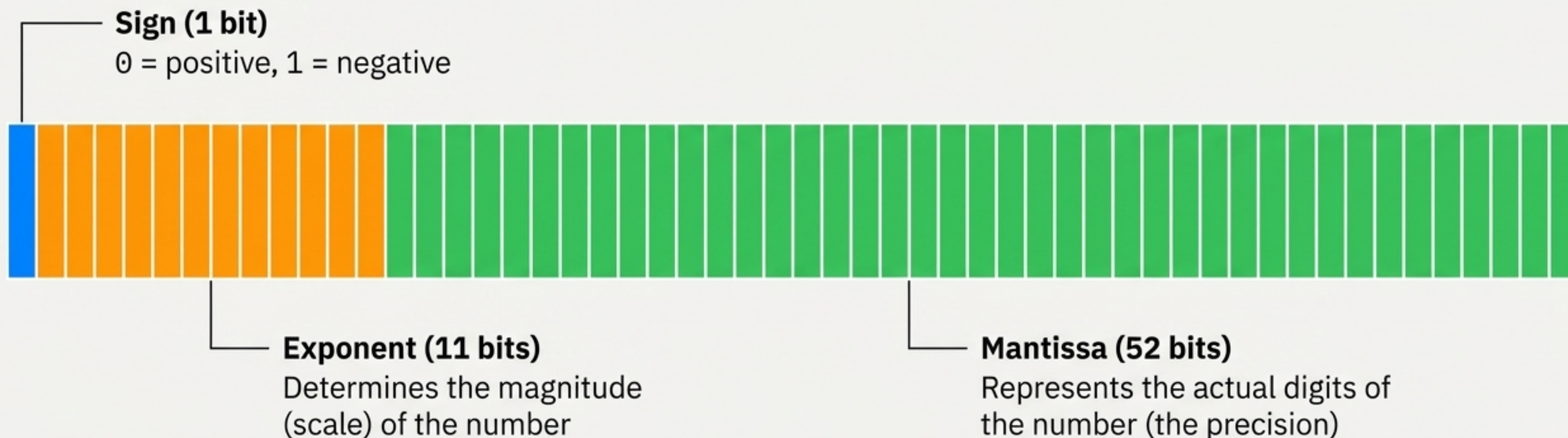
Binary	$111100000000 + .11 = 111100000000.11$
--------	--

Normalized	Move the decimal point 10 places to the left.
------------	---

Scientific Notation	$1.111000000011 * 2^{10}$
---------------------	---------------------------

The Anatomy of a 64-Bit Number

Every JavaScript number is stored in a 64-bit package divided into three parts, each with a specific job.

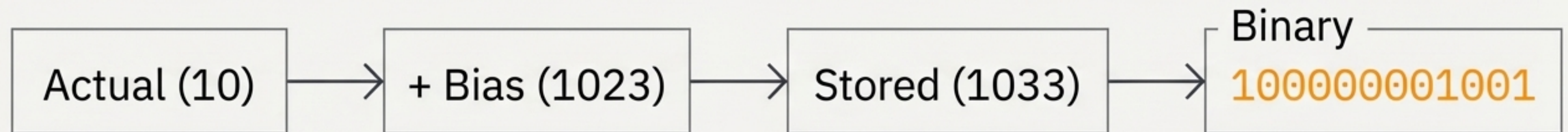


The Biased Exponent: A Clever Trick for Storing Scale

The 11-bit exponent field doesn't store the exponent (10 in our example) directly. Instead, it stores a "biased" value to efficiently represent both positive and negative exponents without needing a separate sign bit.

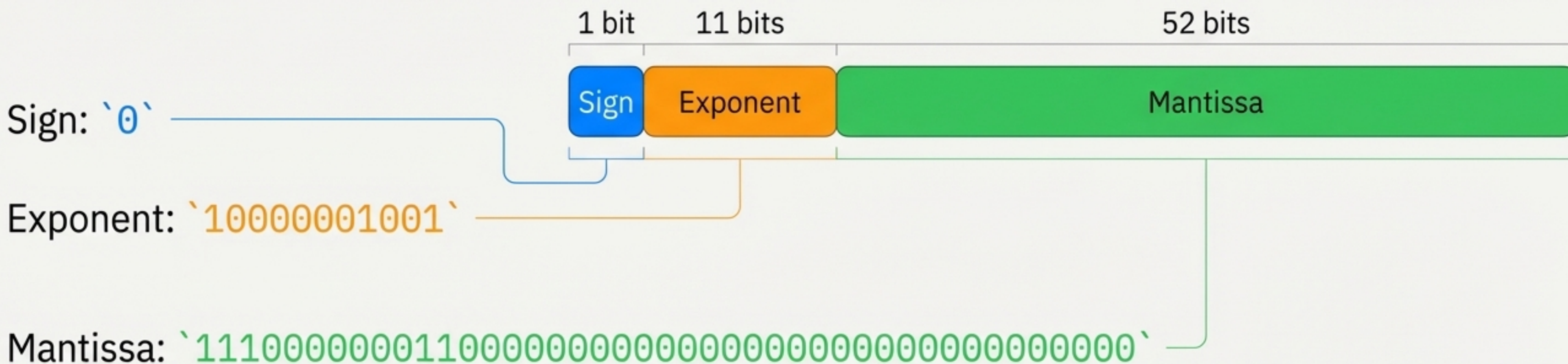
For 64-bit numbers, the bias is 1023.

Formula: $\text{Stored Exponent} = \text{Actual Exponent} + \text{Bias}$



Assembly: Building the Final 64-Bit Representation of 1920.75

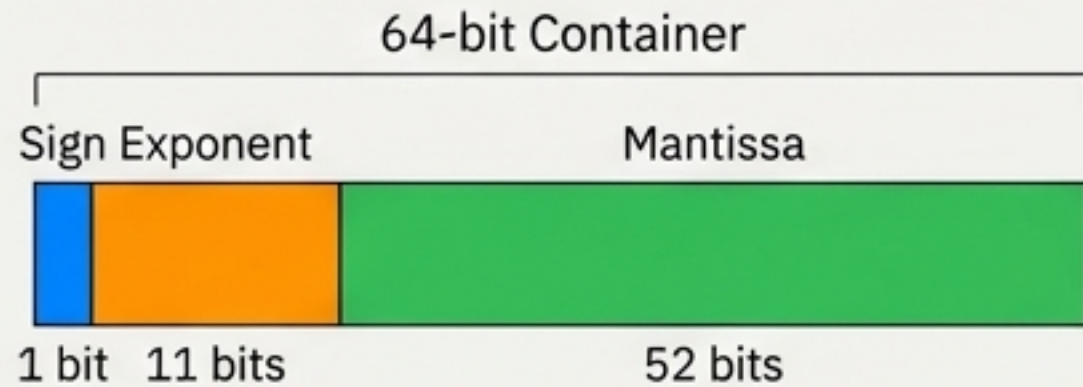
We now have all three components ready to be placed into the 64-bit blueprint.



```
`0 10000001001 1110000000110000000000000000000000000000000`
```


Reversing the Process: The Universal Formula

The IEEE 754 standard defines a precise formula to convert the stored bits back into a decimal number. Notice the `(1 + mantissa)` part—the leading `1` is implied for all normalized numbers, saving a bit of space.



$$Value = (-1)^{\text{sign}} \times (1 + \text{mantissa}) \times 2^{\text{exponent} - 1023}$$

Beyond Finite Numbers: The Special Values.

The IEEE 754 blueprint isn't just for ordinary numbers. By reserving specific patterns for the exponent and mantissa, the standard can elegantly represent concepts like Infinity, Zero, and 'Not a Number' (NaN).



NaN

Representing Infinity.

Infinity is represented when the exponent field is all 1s and the mantissa is all 0s. The the sign bit determines whether it is positive or negative.

+Infinity

Rule: Sign = 0, Exponent = 111...1,
Mantissa = 000...0

01111111111100

-Infinity

Rule: Sign = 1, Exponent = 111...1,
Mantissa = 000...0

111111111000000000000000000000000
00000000000000000000000000000000

Representing Zero and Not a Number (NaN)

Specific patterns are also reserved for **absolute zero** and for the results of invalid mathematical operations.

Zero

Rule: Exponent is all 0s, Mantissa is all 0s.

Reason: Zero cannot be represented with the standard 1.xxx scientific notation.

NaN (Not a Number)

Rule: Exponent is all 1s, Mantissa is anything except all 0s.

Examples: Result of $0 / 0$ or $\text{sqrt}(-1)$.

[illegible]

The Blueprint Empowers the Developer.

JavaScript numbers are not magic—they are a predictable system governed by the IEEE 754 standard.

Understanding this blueprint of Sign, Exponent, and Mantissa explains everything from floating-point precision errors (like ``0.1 + 0.2``) to the behavior of ``Infinity`` and ``NaN``.

Knowing how numbers are built from bits empowers you to write more robust, reliable, and intelligent code.